RTL Design Sherpa

RTL AMBA AXI5-Stream Module Reference — Rev 0.90

July 2026

Table of Contents

1 AXIS5 (AXI5-Stream) Modules

Location: `rtl/amba/axis5/` **Test Location:** `val/amba/` **Status:** Production Ready

---

1.1 Overview

The AXIS5 subsystem gives you AXI5-Stream master and slave endpoints, plus clock-gated variants of each, for high-throughput streaming data applications.

AXI5-Stream extends AXI4-Stream primarily with wake-up signalling (TWAKEUP) for low-power operation. These modules implement the AXI4-Stream handshake and signal set, add TWAKEUP on top, and offer an optional per-byte data parity sideband (TPARITY). That parity signal is an RTL Design Sherpa extension, not an ARM signal — keep that in mind before you wire these endpoints to someone else's IP.

1.2 Implemented Signal Set

Read this table before you integrate with third-party AXI-Stream IP. It's the authoritative statement of what the RTL in `rtl/amba/axis5/` actually carries — not what the ARM spec says could be there.

Signal	Status in these modules	Notes
TDATA	Implemented	Width set by `AXIS_DATA_WIDTH`
TSTRB	Implemented	Width `AXIS_DATA_WIDTH/8`
TKEEP	**Not implemented**	No TKEEP port exists. See "Byte Qualification" below
TLAST	Implemented	Packet/ frame boundary
TID	Implemented	Width `AXIS_ID_WIDTH` (default 8, set 0 to remove)
TDEST	Implemented	Width

Signal	Status in these modules	Notes
		`AXIS_DEST_WIDTH` (default 4, set 0 to remove)
TUSER	Implemented	Width `AXIS_USER_WIDTH` (default 1, set 0 to remove)
TVALID / TREADY	Implemented	Standard handshake
TWAKEUP	Implemented, optional	Gated by `ENABLE_WAKEUP` (default 1)
TPARITY	Implemented, optional	**Not an ARM signal** - RTL Design Sherpa extension, gated by `ENABLE_PARITY` (default 0)

Deviations from the ARM AXI-Stream signal set:

- **No TKEEP.** These modules carry TSTRB only. Null bytes (the TKEEP=0 encoding) cannot be expressed. A stream that needs null-byte signalling has to carry that information in TUSER or use a different endpoint.
- **TPARITY is proprietary.** It occupies no ARM-defined signal name and will not connect to third-party AXI5-Stream IP. Leave `ENABLE_PARITY=0` (the default) for interoperable designs.

- **No TPOISON and no chunking (TCHUNKEN) support.** Neither signal is present in the RTL.
- **TID default width is 8 bits.** `AXIS_ID_WIDTH` is a free parameter, so wider IDs are configurable, but nothing in these modules requires or defaults to a wider ID.

1.3 AXIS4 vs AXIS5 in This Library

This table compares the two generations *as implemented here*, not the full ARM specifications.

Feature	AXIS4 modules (`rtl/amba/axis4/`)	AXIS5 modules (`rtl/amba/axis5/`)
Basic protocol	TVALID/TREADY handshake	TVALID/TREADY handshake
Wake-up	Not present	TWAKEUP, gated by `ENABLE_WAKEUP`
Data parity	Not present	TPARITY sideband plus sticky `parity_error`, gated by `ENABLE_PARITY` (proprietary extension)
Side-band signals	TID, TDEST, TUSER	TID, TDEST, TUSER (same, parameterized widths)
Byte qualification	TSTRB only	TSTRB only
Clock gating	Available (`_cg` variants)	Available (`_cg` variants)
Clock-gate port names	`cfg_cg_enable` / `cfg_cg_idle_count`	**`i_cg_enable`** / **`i_cg_idle_count`** – see below
Stream port prefix on the	`fub_axis_*` / `m_axis_*`	**`fub_axis5_*`** / **`m_axis5_*`** – see below

Feature	AXIS4 modules (`rtl/amba/axis4/`)	AXIS5 modules (`rtl/amba/axis5/`)
`_cg` wrappers		

Two naming outliers, both in the axis5 `_cg` wrappers. Learn them before you port an instantiation across, because they will trip you:

- Thirty-two of the thirty-four `_cg` modules in `rtl/amba` take `cfg_cg_enable`. The two exceptions are `axis5_master_cg` and `axis5_slave_cg`, which take `i_cg_enable` / `i_cg_idle_count`.
- Those same two wrappers carry a `5` in their stream port names (`fub_axis5_tdata`, `m_axis5_tvalid`, …) while the modules they WRAP – `axis5_master`, `axis5_slave` – use `fub_axis_*` / `m_axis_*` / `s_axis_*` without it.

Neither is a defect. Both are inconsistencies that break copy-paste between the axis4 and axis5 pages and between a wrapper and the module inside it. Renaming them is a family-wide interface change and has not been made — so the documentation calls it out instead.

1.4 Module Categories

1.4.1 Stream Master Components

Module	Description	Documentation	Status
axis5_master	AXI5-Stream master for high-throughput data streaming	axis5_master.md	Documented
axis5_master_cg	Clock-gated AXI5-Stream master	axis5_master_cg.md	Documented

1.4.2 Stream Slave Components

Module	Description	Documentation	Status
axis5_slave	AXI5-Stream slave for data reception	axis5_slave.md	Documented
axis5_slave_cg	Clock-gated AXI5-Stream slave	axis5_slave_cg.md	Documented

1.5 Key Features

1.5.1 AXI5-Stream Protocol Support

- **Flow Control:** TVALID/TREADY handshaking with backpressure
- **Packet Boundaries:** TLAST for packet/frame demarcation
- **Byte Qualification:** TSTRB (TKEEP is not implemented - see "Implemented Signal Set")
- **Side-band Data:** TID, TDEST, TUSER for routing and metadata

1.5.2 AXI5 and Extension Features

- **Wake-up Signaling:** TWAKEUP for low-power state exit
- **Optional Data Parity:** TPARITY, one bit per byte, with a sticky `parity_error` flag (proprietary extension)
- **Parameterized Side-band Widths:** TID, TDEST, and TUSER widths are set per instance

1.5.3 Power Management

- **Clock Gating:** Per-module clock gating for power reduction
- **Wake-up Support:** TWAKEUP integration for power management
- **Idle Detection:** Automatic clock gate when stream is idle

1.5.4 Performance

- **Skid-Buffered Path:** All four modules pass data through a `gaxi_skid_buffer`; there is no combinational bypass mode
- **Full-Rate Streaming:** One transfer per clock sustained when downstream keeps TREADY asserted
- **Configurable Widths:** Flexible data and signal widths

1.6 Quick Start

1.6.1 Using AXI5-Stream Master

```
axis5_master #(
    .SKID_DEPTH         (4),
    .AXIS_DATA_WIDTH    (128),
    .AXIS_ID_WIDTH      (8),
    .AXIS_DEST_WIDTH    (4),
    .AXIS_USER_WIDTH    (8),
    .ENABLE_WAKEUP      (1),
```

```
    .ENABLE_PARITY      (0)
) u_axis5_master (
    .aclk               (clk),
    .aresetn            (resetn),

    // FUB streaming interface (from internal logic)
    .fub_axis_tdata     (fub_tdata),
    .fub_axis_tstrb     (fub_tstrb),
    .fub_axis_tlast     (fub_tlast),
    .fub_axis_tid       (fub_tid),
    .fub_axis_tdest     (fub_tdest),
    .fub_axis_tuser     (fub_tuser),
    .fub_axis_tvalid    (fub_tvalid),
    .fub_axis_tready    (fub_tready),
    .fub_axis_twakeup   (fub_twakeup),
    .fub_axis_tparity   ('0),           // Unused when
ENABLE_PARITY=0

    // AXI5-Stream master interface
    .m_axis_tdata       (m_tdata),
    .m_axis_tstrb       (m_tstrb),
    .m_axis_tlast       (m_tlast),
    .m_axis_tid         (m_tid),
    .m_axis_tdest       (m_tdest),
    .m_axis_tuser       (m_tuser),
    .m_axis_tvalid      (m_tvalid),
    .m_axis_tready      (m_tready),
    .m_axis_twakeup     (m_twakeup),
    .m_axis_tparity     (),             // Unused when
ENABLE_PARITY=0

    // Status
    .busy               (m_busy),
    .parity_error       ()              // Unused when
ENABLE_PARITY=0
);
```

1.6.2 Using AXI5-Stream Slave

```
axis5_slave #(
    .SKID_DEPTH         (4),
    .AXIS_DATA_WIDTH    (128),
    .AXIS_ID_WIDTH      (8),
    .AXIS_DEST_WIDTH    (4),
    .AXIS_USER_WIDTH    (8),
    .ENABLE_WAKEUP      (1),
    .ENABLE_PARITY      (0)
) u_axis5_slave (
```

```
    .aclk               (clk),
    .aresetn            (resetn),

    // AXI5-Stream slave interface
    .s_axis_tdata       (s_tdata),
    .s_axis_tstrb       (s_tstrb),
    .s_axis_tlast       (s_tlast),
    .s_axis_tid         (s_tid),
    .s_axis_tdest       (s_tdest),
    .s_axis_tuser       (s_tuser),
    .s_axis_tvalid      (s_tvalid),
    .s_axis_tready      (s_tready),
    .s_axis_twakeup     (s_twakeup),
    .s_axis_tparity     ('0),           // Unused when
ENABLE_PARITY=0

    // FUB streaming interface (to internal logic)
    .fub_axis_tdata     (fub_tdata),
    .fub_axis_tstrb     (fub_tstrb),
    .fub_axis_tlast     (fub_tlast),
    .fub_axis_tid       (fub_tid),
    .fub_axis_tdest     (fub_tdest),
    .fub_axis_tuser     (fub_tuser),
    .fub_axis_tvalid    (fub_tvalid),
    .fub_axis_tready    (fub_tready),
    .fub_axis_twakeup   (fub_twakeup),
    .fub_axis_tparity   (),             // Unused when
ENABLE_PARITY=0

    // Status
    .busy               (s_busy),
    .parity_error       ()              // Unused when
ENABLE_PARITY=0
);
```

1.6.3 Clock-Gated Streaming

```
// Clock-gated master for power efficiency
axis5_master_cg #(
    .AXIS_DATA_WIDTH        (128),
    .CG_IDLE_COUNT_WIDTH    (4)
) u_axis5_master_cg (
    .aclk                   (clk),
    .aresetn                (resetn),

    // Clock gating control
    .i_cg_enable            (1'b1),     // 1 = allow gating, 0 =
clock always on
```

```
    .i_cg_idle_count        (4'd8),      // Idle cycles before the
clock gates

    // Streaming interfaces (note the axis5_ prefix on the _cg
variants)
    .fub_axis5_tdata        (fub_tdata),
    // ... remaining fub_axis5_* ports ...
    .m_axis5_tdata          (m_tdata),
    // ... remaining m_axis5_* ports ...

    // Status
    .busy                   (m_busy),
    .parity_error           (),
    .axis_clock_gating      (m_clk_gated)
);
```

Port prefix warning: the clock-gated variants name their streaming ports `fub_axis5_*` and `m_axis5_*` / `s_axis_*`, while the non-gated variants use `fub_axis_*` and `m_axis_*` / `s_axis_*`. Swapping a module for its `_cg` counterpart therefore means renaming every FUB-side and master-side connection — the exact kind of mechanical edit that's easy to get half-done. See each module page for the exact port list.

1.7 Testing

All AXIS5 modules are verified with CocoTB-based testbenches in `val/amba/`:

```
# Run all AXIS5 tests
pytest val/amba/test_axis5*.py -v

# Run specific module tests
pytest val/amba/test_axis5_master.py -v
pytest val/amba/test_axis5_slave.py -v
```

1.8 Protocol Details

1.8.1 AXI5-Stream Signal Descriptions

Signal	Direction	Description
ACLK	Input	Stream clock (`aclk`)
ARESETN	Input	Active-low asynchronous reset (`aresetn`)

Signal	Direction	Description
TDATA	Master to Slave	Stream data payload
TSTRB	Master to Slave	Byte strobes
TLAST	Master to Slave	End of packet/frame indicator
TID	Master to Slave	Stream identifier
TDEST	Master to Slave	Destination routing information
TUSER	Master to Slave	User-defined sideband data
TWAKEUP	Master to Slave	Wake-up signal (AXI5)
TPARITY	Master to Slave	Per-byte data parity (proprietary extension, optional)
TVALID	Master to Slave	Data valid indicator
TREADY	Slave to Master	Ready to accept data

TKEEP is deliberately absent from this table because no AXIS5 module implements it.

1.8.2 Flow Control

AXI5-Stream flow control is a plain valid/ready handshake:

1. **Data Transfer:** Occurs when TVALID and TREADY are both high
2. **Backpressure:** Slave deasserts TREADY to pause stream
3. **Source Stall:** Master deasserts TVALID when no data available
4. **Packet End:** TLAST indicates final beat of packet/frame

1.8.3 Byte Qualification

These modules carry TSTRB only. The full ARM encoding, shown here for reference, needs both signals:

TKEEP	TSTRB	Meaning
1	1	Data byte
1	0	Position byte (placeholder)
0	0	Null byte (not transmitted)
0	1	Reserved

What these modules support: only the TKEEP=1 rows are expressible. Treat TSTRB=1 as a data byte and TSTRB=0 as a position byte. Null bytes cannot be signalled. When connecting to IP that drives TKEEP, tie its TKEEP to all-ones and pass byte validity through TSTRB, or place a converter at the boundary.

1.9 Design Notes

1.9.1 FUB Interface Pattern

FUB stands for Functional Unit Block: the design-internal logic on the far side of the endpoint from the external stream bus. All AXIS5 modules use this pattern: - **fub_axis_*** (or **fub_axis5_*** on the `_cg` variants) signals connect to internal logic - **m_axis_*** or **s_axis_*** signals connect to the external stream bus - A skid buffer sits between the FUB and external interfaces for timing closure. A skid buffer is a small registered elastic buffer (`gaxi_skid_buffer`, depth `SKID_DEPTH`) that absorbs backpressure so TREADY does not have to propagate combinationally across the endpoint.

1.9.2 Migration from AXI4-Stream

AXIS5 modules keep the AXI4-Stream data path intact: - Core protocol unchanged (TVALID/TREADY handshake) - TWAKEUP can be tied to 0 for always-awake operation, or removed entirely with `ENABLE_WAKEUP=0` - TPARITY is off by default (`ENABLE_PARITY=0`) and adds no ports of consequence when disabled - TUSER width can match AXI4-Stream configurations

Neither generation implements TKEEP, so a design already using the AXIS4 modules will not gain or lose byte-qualification capability by moving to AXIS5.

1.9.3 Streaming Pipeline Integration

AXIS5 modules drop cleanly into streaming pipelines:

```
// Streaming data processing pipeline
axis5_master u_input (
    .m_axis_*(stage1_axis)
);

// Processing stage
processing_block u_process (
    .s_axis_*(stage1_axis),
    .m_axis_*(stage2_axis)
);

// Clock domain crossing
gaxi_fifo_async u_cdc (
```

```
    .s_*(stage2_axis),
    .m_*(stage3_axis)
);

axis5_slave u_output (
    .s_axis_*(stage3_axis)
);
```

1.10 Performance Characteristics

Treat the figures below as design targets, not measured silicon or post-route results. No synthesis or timing run for these modules is published in this repository, and achievable frequency depends heavily on data width, technology, and the surrounding logic. Order-of-magnitude guidance only — quote them in a design review at your own risk.

Metric	Value	Basis
Throughput	1 transfer per clock	Structural - skid buffer sustains full rate while TREADY is high
Latency (skid buffer)	1-2 clock cycles	Registered path from input to output
Backpressure response	1 clock cycle	TREADY deassertion is absorbed by the skid buffer
Maximum frequency	Design target only	Not characterized; no area or Fmax data published

1.11 Related Documentation

- **AXIS4 Modules** - AXI4-Stream components
- **AXI5 Modules** - AXI5 protocol components
- **APB5 Modules** - APB5 protocol components
- **GAXI Modules** - Generic AXI utilities (FIFOs, CDC)

1.12 References

1.12.1 Specifications

- ARM AMBA 5 AXI-Stream Protocol Specification
- ARM AMBA AXI4-Stream Protocol Specification

1.12.2 Source Code

- RTL: `rtl/amba/axis5/`
- Tests: `val/amba/test_axis5*.py`
- Framework: `bin/TBClasses/components/axis4/`

Last Updated: 2026-07-19

1.13 Navigation

- **Back to rtl-amba Index**
- **Back to Main Documentation Index**

2 AXIS5 Master

Module: `axis5_master.sv` **Location:** `rtl/amba/axis5/` **Status:** Production Ready

2.1 Overview

The AXIS5 Master implements an AXI5-Stream master interface with the AMBA5 extensions — wake-up signaling for power management — plus an optional parity sideband for data integrity. Buffering comes from an internal skid buffer, which is what keeps the timing clean at the endpoint boundary.

2.1.1 Key Features

- AXI4-Stream data path (TDATA, TSTRB, TLAST, TID, TDEST, TUSER, TVALID/TREADY)
- TWAKEUP: Wake-up signaling for power management
- TPARITY: Optional parity protection, 1 bit per byte (proprietary extension)
- Internal skid buffer for backpressure handling

- Configurable data, ID, destination, and user signal widths
- Parity error detection and reporting
- Busy status indication

Signal coverage: this module does not implement TKEEP, TPOISON, or any chunking sideband. See Implemented Signal Set for the full list of deviations from the ARM signal set.

2.1.2 AXIS5 Extensions Over AXIS4

Feature	AXIS4	AXIS5	ARM standard signal?
Wake-up signal	None	TWAKEUP (configurable)	Yes, AXI5-Stream addition
Data parity	None	TPARITY (1 bit per byte, configurable)	No - RTL Design Sherpa extension
Parity error detection	None	Built-in sticky `parity_error` flag	No - implementation status output

2.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH	int	4	Internal skid buffer depth
AXIS_DATA_WIDTH	int	32	AXIS data bus width (must be multiple of 8)
AXIS_ID_WIDTH	int	8	AXIS ID signal width (0 to disable)
AXIS_DEST_WIDTH	int	4	AXIS TDEST signal width (0 to disable)
AXIS_USER_WIDTH	int	1	AXIS TUSER signal width (0 to disable)
ENABLE_WAKEUP	bit	1	Enable TWAKEUP signal

Parameter	Type	Default	Description
			(1=enabled)
ENABLE_PARITY	bit	0	Enable TPARITY signal (1=enabled)

Note on `ENABLE_WAKEUP`: the default is 1, so TWAKEUP ports exist and are carried through the buffer unless you explicitly set `ENABLE_WAKEUP=0`. Set it to 0 for an AXI4-Stream-compatible port list with no wake-up sideband and slightly less area. `ENABLE_PARITY` defaults to 0 because TPARITY is a proprietary extension.

2.2.1 Derived Values (do not override)

These are declared in the parameter list so they can be used in port widths, but they are derived from the parameters above. Overriding them directly produces an inconsistent module.

Name	Derivation	Meaning
DW	AXIS_DATA_WIDTH	Data width short name
IW	AXIS_ID_WIDTH	ID width short name
DESTW	AXIS_DEST_WIDTH	DEST width short name
UW	AXIS_USER_WIDTH	USER width short name
SW	DW/8	Strobe width in bytes
PW	SW	Parity width - 1 bit per byte
IW_WIDTH / DESTW_WIDTH / UW_WIDTH	max(width, 1)	Zero-width avoidance for disabled sidebands
PW_WIDTH	ENABLE_PARITY ? PW : 1	TPARITY port width
TSize	Sum of all enabled fields	Skid buffer payload width

2.3 Ports

2.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXIS clock
aresetn	1	Input	AXIS active-low

Port	Width	Direction	Description
			asynchronous reset

2.3.2 FUB AXIS5 Interface (Input Side)

Port	Width	Direction	Description
fub_axis_tdata	DW	Input	Transfer data
fub_axis_tstrb	SW	Input	Transfer byte strobes
fub_axis_tlast	1	Input	Last transfer in packet
fub_axis_tid	IW_WIDTH	Input	Transfer ID (optional)
fub_axis_tdest	DESTW_WIDTH	Input	Transfer destination (optional)
fub_axis_tuser	UW_WIDTH	Input	Transfer user-defined signals (optional)
fub_axis_tvalid	1	Input	Transfer valid
fub_axis_tready	1	Output	Transfer ready (skid buffer not full)
fub_axis_twakeup	1	Input	Wake-up signal (AXIS5 extension)
fub_axis_tparity	PW_WIDTH	Input	Data parity per byte (AXIS5 extension)

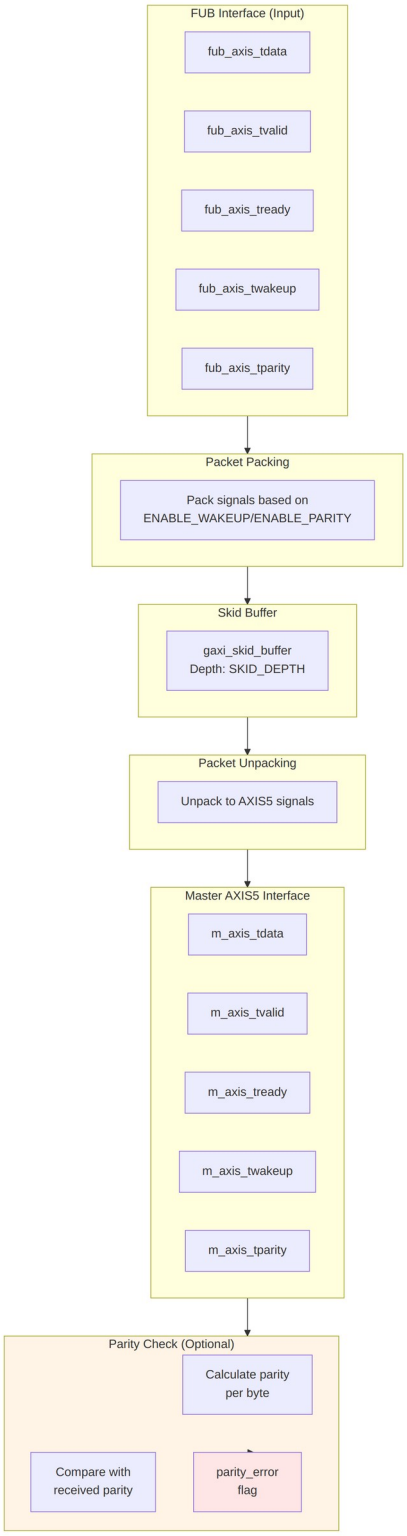
2.3.3 Master AXIS5 Interface (Output Side)

Port	Width	Direction	Description
m_axis_tdata	DW	Output	Transfer data
m_axis_tstrb	SW	Output	Transfer byte strobes
m_axis_tlast	1	Output	Last transfer in packet

Port	Width	Direction	Description
m_axis_tid	IW_WIDTH	Output	Transfer ID (optional)
m_axis_tdest	DESTW_WIDTH	Output	Transfer destination (optional)
m_axis_tuser	UW_WIDTH	Output	Transfer user-defined signals (optional)
m_axis_tvalid	1	Output	Transfer valid
m_axis_tready	1	Input	Transfer ready from downstream
m_axis_twakeup	1	Output	Wake-up signal (AXIS5 extension)
m_axis_tparity	PW_WIDTH	Output	Data parity per byte (AXIS5 extension)

2.3.4 Status Outputs

Port	Width	Direction	Description
busy	1	Output	Module busy (data in buffer or input valid)
parity_error	1	Output	Parity error detected (sticky flag)



Functional Description

2.3.5 Skid Buffer Operation

The module uses an internal `gaxi_skid_buffer` to: - Accept incoming transfers even when downstream is not ready - Prevent backpressure propagation - Provide registered outputs for timing closure - Track buffer occupancy via `busy` signal

2.3.6 Packet Packing/Unpacking

Conditional packing based on configuration:

```
// Full feature set (ENABLE_WAKEUP=1, ENABLE_PARITY=1)
{tdata, tstrb, tlast, tid, tdest, tuser, twakeup, tparity}

// Wake-up only (ENABLE_WAKEUP=1, ENABLE_PARITY=0)
{tdata, tstrb, tlast, tid, tdest, tuser, twakeup}

// Parity only (ENABLE_WAKEUP=0, ENABLE_PARITY=1)
{tdata, tstrb, tlast, tid, tdest, tuser, tparity}

// Base AXIS4 (ENABLE_WAKEUP=0, ENABLE_PARITY=0)
{tdata, tstrb, tlast, tid, tdest, tuser}
```

2.3.7 Parity Checking (Optional)

When `ENABLE_PARITY=1`: 1. Calculate **even** parity for each data byte: `parity[i] = ^m_axis_tdata[i*8 +: 8]`. The XOR reduction yields 1 for an odd number of set bits, so a correct byte plus its parity bit always has an even population count. 2. Compare the calculated parity with `m_axis_tparity` 3. Set `parity_error` flag on mismatch (sticky, cleared only by reset) 4. The check is sampled on accepted output transfers (`m_axis_tvalid && m_axis_tready`)

This module does not generate parity. TPARITY is supplied by the upstream FUB on `fub_axis_tparity`, carried through the skid buffer alongside TDATA, and presented on `m_axis_tparity`. The check on the output side therefore validates the data path *through this module* — packing, buffering, unpacking — against the parity the producer computed. It is not an end-to-end check of the downstream link; the receiving endpoint has to run its own check, which is exactly what `axis5_slave` does on its input side.

If the upstream FUB does not compute parity, leave `ENABLE_PARITY=0`. Tying `fub_axis_tparity` to a constant while parity is enabled will assert `parity_error` on the first non-matching beat — that's the trap people fall into.

2.3.8 Busy Signal

The `busy` output indicates: - Input side has valid data (`fub_axis_tvalid`) - Skid buffer contains data (`int_t_count > 0`)

2.4 Timing Characteristics

2.4.1 Basic Transfer with Wake-up

Timing diagram pending. The signals and sequence this scenario exercises:

- aclk
- fub_axis_tvalid/tready
- fub_axis_tdata
- fub_axis_tlast
- fub_axis_twakeup (AXIS5 extension)
- m_axis_tvalid/tready
- m_axis_tdata
- m_axis_twakeup
- busy

2.4.2 Transfer with Parity Error

Timing diagram pending. The signals and sequence this scenario exercises:

- aclk
- m_axis_tdata
- m_axis_tparity (received)
- calculated_parity
- parity_mismatch
- parity_error (sticky flag)

2.4.3 Skid Buffer Backpressure

Timing diagram pending. The signals and sequence this scenario exercises:

- aclk
- fub_axis_tvalid/tready
- m_axis_tvalid/tready (downstream blocked)
- int_t_count (buffer fill level)
- busy

2.5 Usage Examples

2.5.1 Basic Configuration

```
axis5_master #(
    .SKID_DEPTH       (4),
    .AXIS_DATA_WIDTH  (64),
    .AXIS_ID_WIDTH    (8),
    .AXIS_DEST_WIDTH  (4),
    .AXIS_USER_WIDTH  (1),
    .ENABLE_WAKEUP    (1),
    .ENABLE_PARITY    (0)
) u_axis5_master (
    .aclk               (axis_clk),
    .aresetn            (axis_rst_n),

    // FUB interface (from upstream)
    .fub_axis_tdata     (fub_tdata),
    .fub_axis_tstrb     (fub_tstrb),
    .fub_axis_tlast     (fub_tlast),
    .fub_axis_tid       (fub_tid),
    .fub_axis_tdest     (fub_tdest),
    .fub_axis_tuser     (fub_tuser),
    .fub_axis_tvalid    (fub_tvalid),
    .fub_axis_tready    (fub_tready),
    .fub_axis_twakeup   (fub_twakeup),
    .fub_axis_tparity   (8'h00),  // Not used when ENABLE_PARITY=0

    // Master AXIS5 interface (to downstream)
    .m_axis_tdata       (m_axis_tdata),
    .m_axis_tstrb       (m_axis_tstrb),
    .m_axis_tlast       (m_axis_tlast),
    .m_axis_tid         (m_axis_tid),
    .m_axis_tdest       (m_axis_tdest),
    .m_axis_tuser       (m_axis_tuser),
    .m_axis_tvalid      (m_axis_tvalid),
    .m_axis_tready      (m_axis_tready),
    .m_axis_twakeup     (m_axis_twakeup),
    .m_axis_tparity     (),  // Not used when ENABLE_PARITY=0

    // Status
    .busy               (axis_busy),
    .parity_error       ()  // Not used when ENABLE_PARITY=0
);
```

2.5.2 With Parity Protection

```
axis5_master #(
    .AXIS_DATA_WIDTH  (32),
    .ENABLE_WAKEUP    (1),
    .ENABLE_PARITY    (1)  // Enable parity checking
) u_axis5_master_parity (
    .aclk                (axis_clk),
    .aresetn             (axis_rst_n),

    // FUB interface with parity
    .fub_axis_tdata      (fub_tdata),
    .fub_axis_tstrb      (fub_tstrb),
    .fub_axis_tlast      (fub_tlast),
    .fub_axis_tid        (fub_tid),
    .fub_axis_tdest      (fub_tdest),
    .fub_axis_tuser      (fub_tuser),
    .fub_axis_tvalid     (fub_tvalid),
    .fub_axis_tready     (fub_tready),
    .fub_axis_twakeup    (fub_twakeup),
    .fub_axis_tparity    (fub_tparity),  // 4 bits for 32-bit data

    // Master AXIS5 interface
    .m_axis_tdata        (m_axis_tdata),
    .m_axis_tstrb        (m_axis_tstrb),
    .m_axis_tlast        (m_axis_tlast),
    .m_axis_tid          (m_axis_tid),
    .m_axis_tdest        (m_axis_tdest),
    .m_axis_tuser        (m_axis_tuser),
    .m_axis_tvalid       (m_axis_tvalid),
    .m_axis_tready       (m_axis_tready),
    .m_axis_twakeup      (m_axis_twakeup),
    .m_axis_tparity      (m_axis_tparity),

    // Status - monitor parity errors
    .busy                (axis_busy),
    .parity_error        (axis_parity_err)  // Sticky error flag
);

// Error handling
always_ff @(posedge axis_clk or negedge axis_rst_n) begin
    if (!axis_rst_n)
        error_count <= '0;
    else if (axis_parity_err && !prev_parity_err)
        error_count <= error_count + 1;
end
```

2.6 Design Notes

2.6.1 AXIS5 vs AXIS4 Differences

Feature	AXIS4	AXIS5
Wake-up signal	Not present	TWAKEUP (optional)
Data parity	Not present	TPARITY per byte (optional, proprietary)
Power management	Limited	Enhanced via TWAKEUP
Data integrity	CRC/checksum in TUSER	Built-in parity option
TKEEP	Not present	Not present

2.6.2 Skid Buffer Sizing

- **Typical depth:** 4-8 entries
- Deeper buffers:
  - Absorb longer downstream stalls
  - Increase area and latency
- Shallower buffers:
  - Lower latency
  - May cause upstream backpressure

Recommendation: depth 4 covers most applications; go deeper if your downstream stalls frequently.

2.6.3 Parity Implementation

When `ENABLE_PARITY=1`: - **Overhead:** 1 bit per data byte, so 12.5% extra wires regardless of bus width (a 512-bit bus adds 64 parity bits) - **Detection:** Odd numbers of bit errors within a byte (even parity); two flipped bits in the same byte are undetected - **Correction:** None - error flag only - **Polarity:** Even parity only; there is no parameter to select odd parity - **Use case:** Low-cost error detection in reliable environments

For stronger protection, use: - CRC in TUSER field - ECC (external module) - End-to-end checksums at packet level

2.6.4 Optional Signal Widths

Setting width parameters to 0 disables optional signals: - `AXIS_ID_WIDTH=0` → TID tied to 0, saves area - `AXIS_DEST_WIDTH=0` → TDEST tied to 0 - `AXIS_USER_WIDTH=0` → TUSER tied to 0

Internal logic uses `IW_WIDTH = (IW > 0) ? IW : 1` to avoid zero-width signals.

2.7 Related Modules

- **AXIS5 Slave** - AXIS5 slave interface
- **AXIS5 Master CG** - Clock-gated variant with power management
- **AXIS5 Slave CG** - Clock-gated slave variant
- **AXIS4 Master** - AXIS4 version for comparison
- **AMBA5 Overview** - AMBA5 specifications and extensions

2.8 Testing

`val/amba/test_axis5_master.py` exercises this module. It collects 3 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_axis5_master.py -v
```

2.9 Navigation

- **← Back to AXIS5 Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

3 AXIS5 Master with Clock Gating

Module: `axis5_master_cg.sv` **Location:** `rtl/amba/axis5/` **Status:** Production Ready

3.1 Overview

The AXIS5 Master CG (Clock Gated) wraps the standard AXIS5 master with the AMBA clock gate controller, giving you an AXI5-Stream master interface that gates its own clock during idle periods. Full AXIS5 functionality survives the wrapping — wake-up signaling, optional parity, the skid buffer, all of it.

3.1.1 Key Features

- Same AXI5-Stream data path as `axis5_master` (see its signal coverage note; TKEEP and TPOISON are not implemented)
- Automatic clock gating during idle periods for power savings
- TWAKEUP: Wake-up signaling integration with clock gate control
- TPARITY: Optional parity protection, 1 bit per byte (proprietary extension)
- Configurable idle count threshold for gating activation
- Internal skid buffer for backpressure handling
- Parity error detection and reporting
- Clock gating status output

3.1.2 Power Management Features

Feature	Implementation	Benefit
Clock gating	Automatic via idle detection	Reduces dynamic power
Wake-up integration	TWAKEUP prevents gating	Preserves responsiveness
Configurable threshold	Programmable idle count	Tune power vs. latency
Activity detection	Multi-source OR logic	Comprehensive coverage

3.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH	int	4	Internal skid buffer depth
AXIS_DATA_WIDTH	int	32	AXIS data bus width (must be multiple of 8)

Parameter	Type	Default	Description
AXIS_ID_WIDTH	int	8	AXIS ID signal width (0 to disable)
AXIS_DEST_WIDTH	int	4	AXIS TDEST signal width (0 to disable)
AXIS_USER_WIDTH	int	1	AXIS TUSER signal width (0 to disable)
ENABLE_WAKEUP	bit	1	Enable TWAKEUP signal (1=enabled)
ENABLE_PARITY	bit	0	Enable TPARITY signal (1=enabled)
CG_IDLE_COUNT_WIDTH	int	4	Clock gate idle counter width

Note on `ENABLE_WAKEUP`: the default is 1, so TWAKEUP ports exist and are carried through the buffer unless you explicitly set `ENABLE_WAKEUP=0`. Set it to 0 for an AXI4-Stream-compatible port list with no wake-up sideband and slightly less area. `ENABLE_PARITY` defaults to 0 because TPARITY is a proprietary extension.

3.2.1 Derived Values (do not override)

These are declared in the parameter list so they can be used in port widths, but they are derived from the parameters above. Overriding them directly produces an inconsistent module.

Name	Derivation	Meaning
DW	AXIS_DATA_WIDTH	Data width short name
IW	AXIS_ID_WIDTH	ID width short name
DESTW	AXIS_DEST_WIDTH	DEST width short name
UW	AXIS_USER_WIDTH	USER width short name
SW	DW/8	Strobe width in bytes
PW	SW	Parity width - 1 bit per byte
ICW	CG_IDLE_COUNT_WIDTH	Idle count width short name
IW_WIDTH / DESTW_WIDTH / UW_WIDTH	max(width, 1)	Zero-width avoidance for disabled sidebands

Name	Derivation	Meaning
PW_WIDTH	ENABLE_PARITY ? PW : 1	TPARITY port width

3.3 Ports

3.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXIS ungated clock (always running)
aresetn	1	Input	AXIS active-low asynchronous reset

3.3.2 Clock Gating Configuration

Port	Width	Direction	Description
i_cg_enable	1	Input	Clock gating enable (1=allow gating, 0=force always on)
i_cg_idle_count	ICW	Input	Idle clock cycles before gating activates

3.3.3 FUB AXIS5 Interface (Input Side)

Port	Width	Direction	Description
fub_axis5_tdata	DW	Input	Transfer data
fub_axis5_tstrb	SW	Input	Transfer byte strobes
fub_axis5_tlast	1	Input	Last transfer in packet
fub_axis5_tid	IW_WIDTH	Input	Transfer ID (optional)
fub_axis5_tdest	DESTW_WIDTH	Input	Transfer destination (optional)

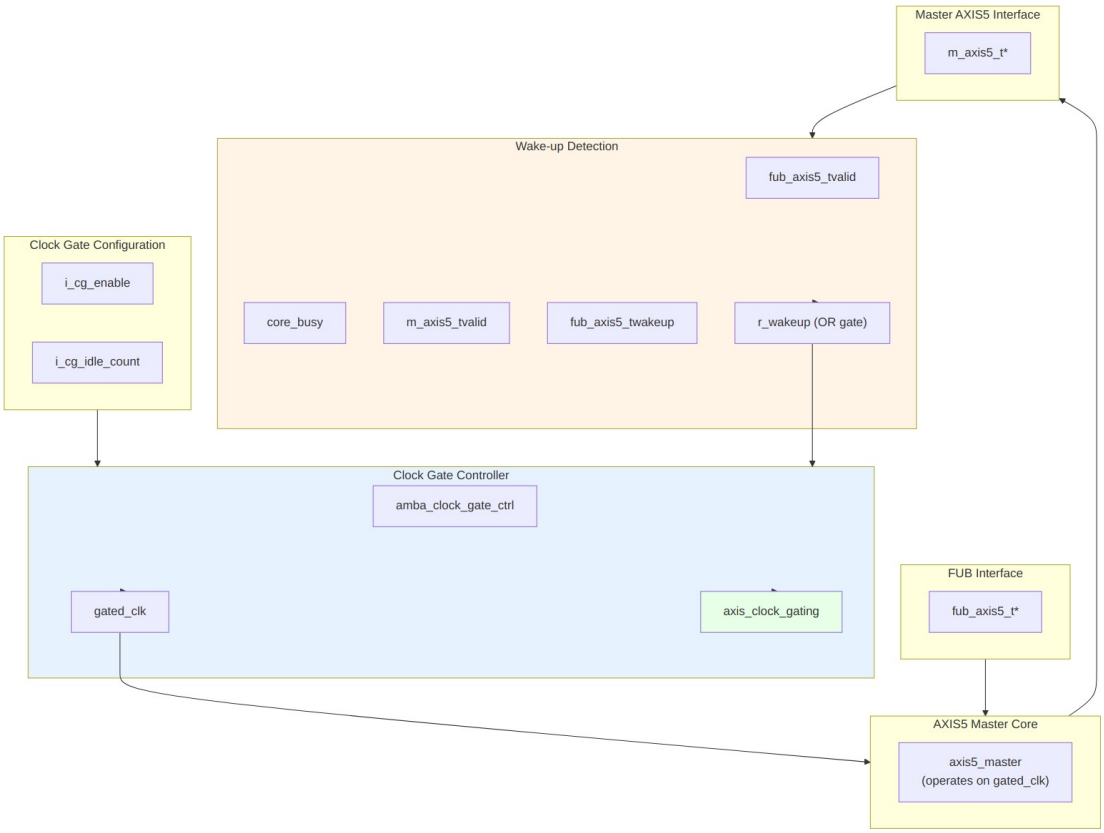
Port	Width	Direction	Description
fub_axis5_tuser	UW_WIDTH	Input	Transfer user-defined signals (optional)
fub_axis5_tvalid	1	Input	Transfer valid
fub_axis5_tready	1	Output	Transfer ready (skid buffer not full)
fub_axis5_twakeup	1	Input	Wake-up signal (prevents clock gating)
fub_axis5_tparity	PW_WIDTH	Input	Data parity per byte (AXIS5 extension)

3.3.4 Master AXIS5 Interface (Output Side)

Port	Width	Direction	Description
m_axis5_tdata	DW	Output	Transfer data
m_axis5_tstrb	SW	Output	Transfer byte strobes
m_axis5_tlast	1	Output	Last transfer in packet
m_axis5_tid	IW_WIDTH	Output	Transfer ID (optional)
m_axis5_tdest	DESTW_WIDTH	Output	Transfer destination (optional)
m_axis5_tuser	UW_WIDTH	Output	Transfer user-defined signals (optional)
m_axis5_tvalid	1	Output	Transfer valid
m_axis5_tready	1	Input	Transfer ready from downstream
m_axis5_twakeup	1	Output	Wake-up signal
m_axis5_tparity	PW_WIDTH	Output	Data parity per byte (AXIS5 extension)

3.3.5 Status Outputs

Port	Width	Direction	Description
busy	1	Output	Module busy (data in buffer or input valid)
parity_error	1	Output	Parity error detected (sticky flag)
axis_clock_gating	1	Output	Clock gating active status (1=gated, 0=running)



Functional Description

3.3.6 Clock Gating Control

Wake-up detection logic:

```
r_wakeup <= fub_axis5_tvalid ||  // Input has data
            core_busy ||          // Core processing
            m_axis5_tvalid ||     // Output has data
            fub_axis5_twakeup;    // Explicit wake-up
```

The clock gate controller: 1. Monitors `r_wakeup` for activity 2. If idle (r_wakeup=0) for `i_cg_idle_count` cycles, gates the clock 3. On activity (r_wakeup=1), ungates the clock 4. Respects `i_cg_enable` (clock forced on when 0)

3.3.7 Wake-up and Gating Latency

Activity is registered once (AXI4, AXI5, AXI4-Lite, AXI4-Stream) or twice (APB, APB5, AXI5-Stream – with one exception: `apb4_slave_cdc_cg` drives `amba_clock_gate_ctrl` combinationally and so registers once, not twice) before reaching the ICG enable, which is combinational. AXI5-Stream is a **two-stage** family: `r_wakeup` in this wrapper, then a second `r_wakeup` inside `amba_clock_gate_ctrl`. The first gated-clock rising edge available to the core therefore arrives 3 `aclk` cycles after activity asserts. Budget accordingly:

Event	Latency	Path
Activity to first usable gated-clock edge	3 `aclk` cycles (2 register stages)	`fub_axis5_tvalid` to `r_wakeup` to controller `r_wakeup` to combinational ICG enable to next edge
Last activity to clock gated	`i_cg_idle_count` + 3 `aclk` cycles	`i_cg_idle_count` + 1 after the internal wakeup deasserts, plus two wake-up register stages

The wake-up path is **not** zero-latency. Nothing is lost during those cycles: `fub_axis5_tready` is driven by the skid buffer on `gated_clk`, so it stays low while the clock is stopped and the producer simply holds TVALID until the clock resumes. Both registers and the idle counter run on the ungated `aclk`, so the wake-up path stays live even while the core clock is stopped.

3.3.8 Reset Behavior

Reset is safe with respect to gating; the clock is guaranteed to be running whenever reset is asserted:

- `aresetn` is asynchronous and is passed **ungated** to both the clock gate controller and the core, so its assertion edge does not depend on the gated clock.
- `r_wakeup` in this wrapper and `r_wakeup` inside `amba_clock_gate_ctrl` both reset to 1, which forces the ICG enable active. The clock therefore runs during reset and for at least the idle countdown after reset deassertion.

- `clock_gate_ctrl` loads its idle counter with `i_cg_idle_count` on reset, so gating cannot re-engage until a full idle period has elapsed after release.

Because reset removal is asynchronous, apply the usual practice of releasing `aresetn` synchronously to `aclk` at the system level.

3.3.9 Power Saving Mechanism

Clock gating states:

Condition	r_wakeup	Clock State	Power State
Active transfer	1	Running	Full power
Buffer has data	1	Running	Full power
Wake-up asserted	1	Running	Full power
Idle < threshold	0	Running	Full power
Idle ≥ threshold	0	Gated	Low power

Benefits: - Reduces dynamic power during idle periods - Wake-up on new transfers costs 2 register stages; the first usable gated-clock edge arrives 3 `aclk` cycles after activity (see Wake-up and Gating Latency) - No protocol impact (transparent to upstream/downstream)

3.3.10 Integration with AXIS5 Extensions

Wake-up signal dual purpose: 1. **Protocol:** Power management hint to downstream 2. **Clock gating:** Prevents local clock gating

This dual use is what guarantees wake-up events are never missed because the clock happened to be gated.

3.3.11 Idle Count Configuration

Typical values for `i_cg_idle_count`: - **Aggressive power saving:** 2-4 cycles - Quick gating, may have frequent gate/ungate transitions - Best for bursty traffic with long idle periods - **Balanced:** 8-16 cycles - Reduces gate/ungate overhead - Good for moderate traffic patterns - **Conservative:** 32-64 cycles - Only gates during extended idle - Minimal gate/ungate overhead

Trade-off: lower count means more power savings but more gate transitions, and each transition has its own dynamic overhead.

3.4 Timing Characteristics

3.4.1 Clock Gating Activation

Timing diagram pending. The signals and sequence this scenario exercises:

- aclk (always running)
- r_wakeup (activity detection)
- i_cg_idle_count (threshold)
- gated_clk (starts gating after threshold)
- axis_clock_gating (status)
- fub_axis5_tvalid (new transfer wakes up)

3.4.2 Wake-up Signal Preventing Gating

Timing diagram pending. The signals and sequence this scenario exercises:

- aclk
- fub_axis5_twakeup (asserted)
- r_wakeup (stays high)
- gated_clk (remains running)
- axis_clock_gating (stays 0)

3.4.3 Transfer During Idle Period

Timing diagram pending. The signals and sequence this scenario exercises:

- aclk
- gated_clk (gated, then ungates on transfer)
- fub_axis5_tvalid (new transfer)
- r_wakeup (0 → 1 transition)
- axis_clock_gating (1 → 0)
- m_axis5_tvalid (output after ungate)

3.5 Usage Examples

3.5.1 Basic Clock-Gated Configuration

```
axis5_master_cg #(
    .SKID_DEPTH             (4),
    .AXIS_DATA_WIDTH        (64),
    .AXIS_ID_WIDTH          (8),
    .AXIS_DEST_WIDTH        (4),
    .AXIS_USER_WIDTH        (1),
    .ENABLE_WAKEUP          (1),
    .ENABLE_PARITY          (0),
    .CG_IDLE_COUNT_WIDTH    (4)
) u_axis5_master_cg (
    .aclk                   (axis_clk),
    .aresetn                (axis_rst_n),

    // Clock gating configuration
    .i_cg_enable            (1'b1),        // Enable clock gating
    .i_cg_idle_count        (4'd8),        // Gate after 8 idle cycles

    // FUB interface (from upstream)
    .fub_axis5_tdata        (fub_tdata),
    .fub_axis5_tstrb        (fub_tstrb),
    .fub_axis5_tlast        (fub_tlast),
    .fub_axis5_tid          (fub_tid),
    .fub_axis5_tdest        (fub_tdest),
    .fub_axis5_tuser        (fub_tuser),
    .fub_axis5_tvalid       (fub_tvalid),
    .fub_axis5_tready       (fub_tready),
    .fub_axis5_twakeup      (fub_twakeup),
    .fub_axis5_tparity      (8'h00),

    // Master AXIS5 interface (to downstream)
    .m_axis5_tdata          (m_axis_tdata),
    .m_axis5_tstrb          (m_axis_tstrb),
    .m_axis5_tlast          (m_axis_tlast),
    .m_axis5_tid            (m_axis_tid),
    .m_axis5_tdest          (m_axis_tdest),
    .m_axis5_tuser          (m_axis_tuser),
    .m_axis5_tvalid         (m_axis_tvalid),
    .m_axis5_tready         (m_axis_tready),
    .m_axis5_twakeup        (m_axis_twakeup),
    .m_axis5_tparity        (),

    // Status
    .busy                   (axis_busy),
```

```
    .parity_error           (),
    .axis_clock_gating      (axis_clk_gated)  // Monitor gating status
);
```

3.5.2 Dynamic Clock Gating Control

```
// Runtime control of clock gating
logic cg_enable;
logic [3:0] cg_idle_count;

axis5_master_cg #(
    .AXIS_DATA_WIDTH        (32),
    .ENABLE_WAKEUP          (1),
    .ENABLE_PARITY          (1),
    .CG_IDLE_COUNT_WIDTH    (4)
) u_axis5_master_cg (
    .aclk                   (sys_clk),
    .aresetn                (sys_rst_n),

    // Dynamic configuration via registers
    .i_cg_enable            (cg_enable),
    .i_cg_idle_count        (cg_idle_count),

    // ... ports ...

    .busy                   (axis_busy),
    .parity_error           (parity_err),
    .axis_clock_gating      (clk_gated_status)
);

// Configuration register interface
always_ff @(posedge cfg_clk or negedge cfg_rst_n) begin
    if (!cfg_rst_n) begin
        cg_enable <= 1'b1;      // Default: gating enabled
        cg_idle_count <= 4'd16; // Default: 16 cycle threshold
    end else if (cfg_write) begin
        case (cfg_addr)
            ADDR_CG_CTRL: cg_enable <= cfg_wdata[0];
            ADDR_CG_IDLE: cg_idle_count <= cfg_wdata[3:0];
        endcase
    end
end

// Monitor power savings
always_ff @(posedge sys_clk or negedge sys_rst_n) begin
    if (!sys_rst_n)
        gated_cycle_count <= '0;
```

```
    else if (clk_gated_status)
        gated_cycle_count <= gated_cycle_count + 1;
end
```

3.5.3 Power Profiling Example

```
// Instantiate both gated and non-gated versions for comparison
axis5_master_cg #(
    .AXIS_DATA_WIDTH    (64),
    .CG_IDLE_COUNT_WIDTH(4)
) u_gated (
    .aclk               (clk),
    .aresetn            (rst_n),
    .i_cg_enable        (1'b1),
    .i_cg_idle_count    (4'd8),
    // ... ports ...
    .axis_clock_gating  (gated_status)
);

// Track gating statistics
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        total_cycles <= '0;
        gated_cycles <= '0;
    end else begin
        total_cycles <= total_cycles + 1;
        if (gated_status)
            gated_cycles <= gated_cycles + 1;
    end
end

// Calculate power savings percentage
assign gated_duty_pct = (gated_cycles * 100) / total_cycles;
```

Read this metric carefully. `gated_duty_pct` is the fraction of time the clock was gated, not a power saving. Actual dynamic power saved depends on the switching activity that would have occurred during those cycles; leakage is unaffected by gating; and the gating logic and its clock-tree branch consume power themselves. Use the number as a gating-effectiveness indicator, and get real figures from a power analysis tool fed a VCD or SAIF from a representative workload.

3.6 Design Notes

3.6.1 Clock Gating vs. Non-Gated Variant

Aspect	axis5_master	axis5_master_cg
Area	Smaller	+5-10% (clock gate logic)
Dynamic power	Higher	Lower (gating reduces)
Static power	Same	Same
Latency	Lower	Same in steady state; +3 `aclk` cycles when waking from a gated state (two register stages)
Complexity	Lower	Higher (gating control)
Use case	Always-on systems	Power-sensitive designs

When to use clock-gated variant: - Battery-powered devices - Bursty traffic patterns with idle periods - Power budget constraints - SoC integration with power domains

3.6.2 Wake-up Integration Benefits

TWAKEUP integration with clock gating provides: 1. **Event preservation:** Wake-up events prevent clock gating 2. **Bounded restart:** Clock activation takes a fixed 2 `aclk` cycles, and no transfer is dropped while it happens 3. **Protocol compliance:** Wake-up propagated correctly 4. **Power efficiency:** Only wake when necessary

3.6.3 Gate Transition Overhead

Each clock gate transition has overhead: - **Gate activation:** `i_cg_idle_count` + 1 cycles after the internal wakeup deasserts, which is `i_cg_idle_count` + 3 cycles after the last bus activity - **Ungate activation:** two wake-up register stages; first usable gated-clock edge 3 cycles after activity asserts - **Dynamic power:** Gate/ungate switching consumes power

Recommendation: set `i_cg_idle_count` high enough to amortize the transition overhead.

3.6.4 Clock Domain Considerations

- **Input signals:** Must be on `aclk` domain (always running)
- **Core logic:** Operates on `gated_clk` domain
- **Output signals:** Driven by `gated_clk` domain
- **Status outputs:** `axis_clock_gating` on `aclk` domain

No CDC synchronizers are needed, because `gated_clk` is `aclk` passed through an ICG cell: same frequency, same phase, no independent clock source. There is still physical work to do:

- **Clock tree synthesis:** `gated_clk` must be balanced against `aclk`. The wake-up registers and `axis_clock_gating` sit on `aclk` while the core sits on `gated_clk`, so uncontrolled skew between the two branches eats directly into the timing budget on every path that crosses between them.
- **ICG placement:** place the ICG cell close to the root of the gated branch so the insertion delay it adds is common to the whole core.
- **FPGA targets:** `clock_gate_ctrl` instantiates an ICG cell, which is an ASIC library primitive. On FPGAs use a clock-enable style implementation or a device buffer with an enable (for example `BUFGCE`) instead; a bare ICG will not map cleanly.

3.7 Related Modules

- **AXIS5 Master** - Base AXIS5 master without clock gating
- **AXIS5 Slave CG** - Clock-gated slave variant
- **AXIS5 Slave** - Base AXIS5 slave
- **AMBA Clock Gate Controller** - Clock gating controller
- **AMBA5 Power Management** - AMBA5 power features

3.8 Testing

`val/amba/test_axis5_master_cg.py` exercises this module. It collects 3 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_axis5_master_cg.py -v
```

3.9 Navigation

- **← Back to AXIS5 Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

4 AXIS5 Slave

Module: `axis5_slave.sv` **Location:** `rtl/amba/axis5/` **Status:** Production Ready

4.1 Overview

The AXIS5 Slave implements an AXI5-Stream slave interface with the AMBA5 extensions — wake-up signaling for power management — plus an optional parity sideband for data integrity. It accepts incoming stream data and buffers it through an internal skid buffer, which is what keeps backpressure off your upstream link.

4.1.1 Key Features

- AXI4-Stream data path (TDATA, TSTRB, TLAST, TID, TDEST, TUSER, TVALID/TREADY)
- TWAKEUP: Wake-up signaling for power management
- TPARITY: Optional parity protection, 1 bit per byte (proprietary extension)
- Internal skid buffer for backpressure handling
- Configurable data, ID, destination, and user signal widths
- Parity error detection and reporting
- Busy status indication

Signal coverage: this module does not implement TKEEP, TPOISON, or any chunking sideband. See Implemented Signal Set for the full list of deviations from the ARM signal set.

4.1.2 AXIS5 Extensions Over AXIS4

Feature	AXIS4	AXIS5	ARM standard signal?
Wake-up signal	None	TWAKEUP (configurable)	Yes, AXI5-Stream addition
Data parity	None	TPARITY (1 bit per byte, configurable)	No - RTL Design Sherpa extension
Parity error detection	None	Built-in sticky `parity_error` flag	No - implementation status output

4.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH	int	4	Internal skid buffer depth
AXIS_DATA_WIDTH	int	32	AXIS data bus width (must be multiple of 8)
AXIS_ID_WIDTH	int	8	AXIS ID signal width (0 to disable)
AXIS_DEST_WIDTH	int	4	AXIS TDEST signal width (0 to disable)
AXIS_USER_WIDTH	int	1	AXIS TUSER signal width (0 to disable)
ENABLE_WAKEUP	bit	1	Enable TWAKEUP signal (1=enabled)
ENABLE_PARITY	bit	0	Enable TPARITY signal (1=enabled)

Note on `ENABLE_WAKEUP`: the default is 1, so TWAKEUP ports exist and are carried through the buffer unless you explicitly set `ENABLE_WAKEUP=0`. Set it to 0 for an AXI4-Stream-compatible port list with no wake-up sideband and slightly less area. `ENABLE_PARITY` defaults to 0 because TPARITY is a proprietary extension.

4.2.1 Derived Values (do not override)

These are declared in the parameter list so they can be used in port widths, but they are derived from the parameters above. Overriding them directly produces an inconsistent module.

Name	Derivation	Meaning
DW	AXIS_DATA_WIDTH	Data width short name
IW	AXIS_ID_WIDTH	ID width short name
DESTW	AXIS_DEST_WIDTH	DEST width short name
UW	AXIS_USER_WIDTH	USER width short name
SW	DW/8	Strobe width in bytes
PW	SW	Parity width - 1 bit per byte
IW_WIDTH / DESTW_WIDTH	max(width, 1)	Zero-width avoidance for disabled sidebands

Name	Derivation	Meaning
/ UW_WIDTH		
PW_WIDTH	ENABLE_PARITY ? PW : 1	TPARITY port width
TSize	Sum of all enabled fields	Skid buffer payload width

4.3 Ports

4.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXIS clock
aresetn	1	Input	AXIS active-low asynchronous reset

4.3.2 Slave AXIS5 Interface (Input Side)

Port	Width	Direction	Description
s_axis_tdata	DW	Input	Transfer data from upstream
s_axis_tstrb	SW	Input	Transfer byte strobes
s_axis_tlast	1	Input	Last transfer in packet
s_axis_tid	IW_WIDTH	Input	Transfer ID (optional)
s_axis_tdest	DESTW_WIDTH	Input	Transfer destination (optional)
s_axis_tuser	UW_WIDTH	Input	Transfer user-defined signals (optional)
s_axis_tvalid	1	Input	Transfer valid from upstream
s_axis_tready	1	Output	Transfer ready (skid buffer not full)
s_axis_twakeup	1	Input	Wake-up signal (AXIS5 extension)

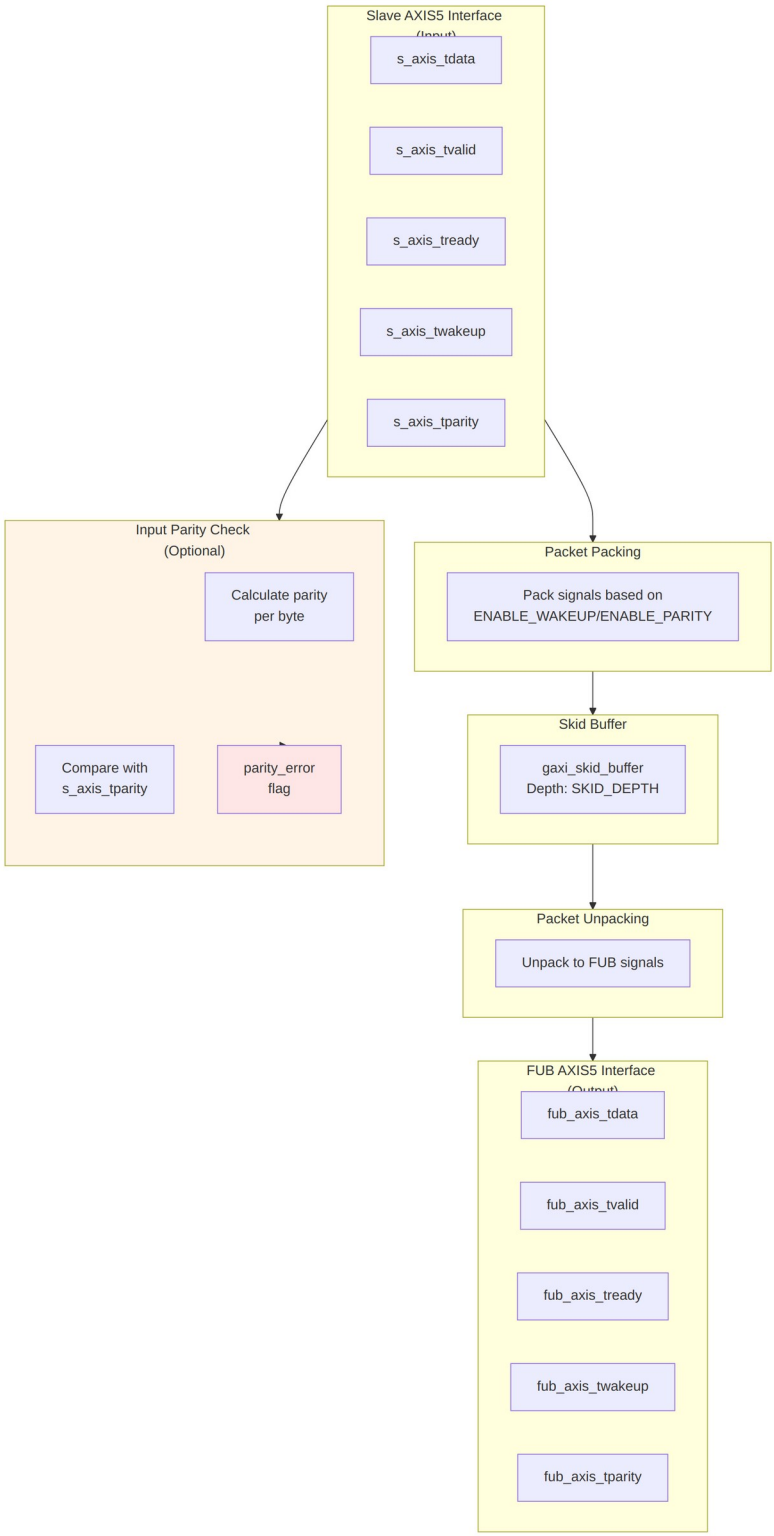
Port	Width	Direction	Description
s_axis_tparity	PW_WIDTH	Input	Data parity per byte (AXIS5 extension)

4.3.3 FUB AXIS5 Interface (Output Side to Backend)

Port	Width	Direction	Description
fub_axis_tdata	DW	Output	Transfer data to backend
fub_axis_tstrb	SW	Output	Transfer byte strobes
fub_axis_tlast	1	Output	Last transfer in packet
fub_axis_tid	IW_WIDTH	Output	Transfer ID (optional)
fub_axis_tdest	DESTW_WIDTH	Output	Transfer destination (optional)
fub_axis_tuser	UW_WIDTH	Output	Transfer user-defined signals (optional)
fub_axis_tvalid	1	Output	Transfer valid to backend
fub_axis_tready	1	Input	Transfer ready from backend
fub_axis_twakeup	1	Output	Wake-up signal (AXIS5 extension)
fub_axis_tparity	PW_WIDTH	Output	Data parity per byte (AXIS5 extension)

4.3.4 Status Outputs

Port	Width	Direction	Description
busy	1	Output	Module busy (data in buffer or input valid)
parity_error	1	Output	Parity error detected on input (sticky flag)



Functional Description

4.3.5 Skid Buffer Operation

The module uses an internal `gaxi_skid_buffer` to: - Accept incoming transfers even when backend is not ready - Prevent backpressure to upstream - Provide registered outputs for timing closure - Track buffer occupancy via `busy` signal

4.3.6 Packet Packing/Unpacking

Conditional packing based on configuration:

```
// Full feature set (ENABLE_WAKEUP=1, ENABLE_PARITY=1)
{tdata, tstrb, tlast, tid, tdest, tuser, twakeup, tparity}

// Wake-up only (ENABLE_WAKEUP=1, ENABLE_PARITY=0)
{tdata, tstrb, tlast, tid, tdest, tuser, twakeup}

// Parity only (ENABLE_WAKEUP=0, ENABLE_PARITY=1)
{tdata, tstrb, tlast, tid, tdest, tuser, tparity}

// Base AXIS4 (ENABLE_WAKEUP=0, ENABLE_PARITY=0)
{tdata, tstrb, tlast, tid, tdest, tuser}
```

4.3.7 Parity Checking (Optional)

When `ENABLE_PARITY=1`: 1. Calculate **even** parity for each input data byte: `parity[i] = ^s_axis_tdata[i*8 +: 8]`. The XOR reduction yields 1 for an odd number of set bits, so a correct byte plus its parity bit always has an even population count. 2. Compare the calculated parity with the received `s_axis_tparity` 3. Set `parity_error` flag on mismatch (sticky, cleared only by reset) 4. Parity check occurs on accepted input transfers (`s_axis_tvalid && s_axis_tready`)

Note: parity checking happens on the **input side**, so it covers the upstream link and catches errors as early as possible. This is the receiver-side check of the pair; `axis5_master` checks on its output side, which covers only its own internal data path.

TPARITY is an RTL Design Sherpa extension, not an ARM AXI5-Stream signal. Leave `ENABLE_PARITY=0` (the default) when interoperating with third-party stream IP.

4.3.8 Busy Signal

The `busy` output indicates: - Input side has valid data (`s_axis_tvalid`) - Skid buffer contains data (`int_t_count > 0`)

4.4 Timing Characteristics

4.4.1 Basic Receive with Wake-up

Timing diagram pending. The signals and sequence this scenario exercises:

- aclk
- s_axis_tvalid/tready
- s_axis_tdata
- s_axis_tlast
- s_axis_twakeup (AXIS5 extension)
- fub_axis_tvalid/tready
- fub_axis_tdata
- fub_axis_twakeup
- busy

4.4.2 Receive with Parity Error

Timing diagram pending. The signals and sequence this scenario exercises:

- aclk
- s_axis_tdata
- s_axis_tparity (received)
- calculated_parity
- parity_mismatch
- parity_error (sticky flag set)

4.4.3 Skid Buffer Backpressure from Backend

Timing diagram pending. The signals and sequence this scenario exercises:

- aclk
- s_axis_tvalid/tready
- fub_axis_tvalid/tready (backend blocked)
- int_t_count (buffer fill level)
- busy

4.5 Usage Examples

4.5.1 Basic Configuration

```
axis5_slave #(
    .SKID_DEPTH       (4),
    .AXIS_DATA_WIDTH  (64),
    .AXIS_ID_WIDTH    (8),
    .AXIS_DEST_WIDTH  (4),
    .AXIS_USER_WIDTH  (1),
    .ENABLE_WAKEUP    (1),
    .ENABLE_PARITY    (0)
) u_axis5_slave (
    .aclk               (axis_clk),
    .aresetn            (axis_rst_n),

    // Slave interface (from upstream)
    .s_axis_tdata       (s_axis_tdata),
    .s_axis_tstrb       (s_axis_tstrb),
    .s_axis_tlast       (s_axis_tlast),
    .s_axis_tid         (s_axis_tid),
    .s_axis_tdest       (s_axis_tdest),
    .s_axis_tuser       (s_axis_tuser),
    .s_axis_tvalid      (s_axis_tvalid),
    .s_axis_tready      (s_axis_tready),
    .s_axis_twakeup     (s_axis_twakeup),
    .s_axis_tparity     (8'h00),  // Not used when ENABLE_PARITY=0

    // FUB interface (to backend)
    .fub_axis_tdata     (fub_tdata),
    .fub_axis_tstrb     (fub_tstrb),
    .fub_axis_tlast     (fub_tlast),
    .fub_axis_tid       (fub_tid),
    .fub_axis_tdest     (fub_tdest),
    .fub_axis_tuser     (fub_tuser),
    .fub_axis_tvalid    (fub_tvalid),
    .fub_axis_tready    (fub_tready),
    .fub_axis_twakeup   (fub_twakeup),
    .fub_axis_tparity   (),  // Not used when ENABLE_PARITY=0

    // Status
    .busy               (axis_busy),
    .parity_error       ()  // Not used when ENABLE_PARITY=0
);
```

4.5.2 With Parity Protection and Error Handling

```
axis5_slave #(
    .AXIS_DATA_WIDTH  (32),
    .ENABLE_WAKEUP    (1),
    .ENABLE_PARITY    (1)  // Enable parity checking on input
) u_axis5_slave_parity (
    .aclk                (axis_clk),
    .aresetn             (axis_rst_n),

    // Slave interface with parity
    .s_axis_tdata        (s_axis_tdata),
    .s_axis_tstrb        (s_axis_tstrb),
    .s_axis_tlast        (s_axis_tlast),
    .s_axis_tid          (s_axis_tid),
    .s_axis_tdest        (s_axis_tdest),
    .s_axis_tuser        (s_axis_tuser),
    .s_axis_tvalid       (s_axis_tvalid),
    .s_axis_tready       (s_axis_tready),
    .s_axis_twakeup      (s_axis_twakeup),
    .s_axis_tparity      (s_axis_tparity),  // 4 bits for 32-bit data

    // FUB interface
    .fub_axis_tdata      (fub_tdata),
    .fub_axis_tstrb      (fub_tstrb),
    .fub_axis_tlast      (fub_tlast),
    .fub_axis_tid        (fub_tid),
    .fub_axis_tdest      (fub_tdest),
    .fub_axis_tuser      (fub_tuser),
    .fub_axis_tvalid     (fub_tvalid),
    .fub_axis_tready     (fub_tready),
    .fub_axis_twakeup    (fub_twakeup),
    .fub_axis_tparity    (fub_tparity),

    // Status - monitor parity errors
    .busy                (axis_busy),
    .parity_error        (axis_parity_err)  // Sticky error flag
);

// Error handling and logging
always_ff @(posedge axis_clk or negedge axis_rst_n) begin
    if (!axis_rst_n) begin
        error_count <= '0;
        prev_parity_err <= 1'b0;
    end else begin
        prev_parity_err <= axis_parity_err;
        // Count rising edges (new errors)
        if (axis_parity_err && !prev_parity_err) begin
```

```
            error_count <= error_count + 1;
            // Log error or trigger interrupt
            $error("AXIS5 Slave: Parity error detected at time %t",
$time);
        end
    end
end
```

4.5.3 Integration with Processing Pipeline

```
// AXIS5 slave receives data from network/upstream
axis5_slave #(
    .AXIS_DATA_WIDTH  (512),  // Wide data path
    .AXIS_ID_WIDTH    (4),
    .ENABLE_WAKEUP    (1),
    .ENABLE_PARITY    (1)
) u_rx_slave (
    .aclk              (sys_clk),
    .aresetn           (sys_rst_n),
    .s_axis_tdata      (rx_tdata),
    .s_axis_tstrb      (rx_tstrb),
    .s_axis_tlast      (rx_tlast),
    .s_axis_tid        (rx_tid),
    .s_axis_tdest      (rx_tdest),
    .s_axis_tuser      (rx_tuser),
    .s_axis_tvalid     (rx_tvalid),
    .s_axis_tready     (rx_tready),
    .s_axis_twakeup    (rx_twakeup),
    .s_axis_tparity    (rx_tparity),
    // To processing pipeline
    .fub_axis_tdata    (proc_tdata),
    .fub_axis_tstrb    (proc_tstrb),
    .fub_axis_tlast    (proc_tlast),
    .fub_axis_tid      (proc_tid),
    .fub_axis_tdest    (proc_tdest),
    .fub_axis_tuser    (proc_tuser),
    .fub_axis_tvalid   (proc_tvalid),
    .fub_axis_tready   (proc_tready),
    .fub_axis_twakeup  (proc_twakeup),
    .fub_axis_tparity  (proc_tparity),
    .busy              (rx_busy),
    .parity_error      (rx_parity_err)
);

// Connect to processing module
my_data_processor u_processor (
    .clk         (sys_clk),
    .rst_n       (sys_rst_n),
    .in_tdata    (proc_tdata),
```

```
    .in_tvalid   (proc_tvalid),
    .in_tready   (proc_tready),
    // ... other signals
);
```

4.6 Design Notes

4.6.1 AXIS5 vs AXIS4 Differences

Feature	AXIS4	AXIS5
Wake-up signal	Not supported	TWAKEUP (optional)
Data parity	Not supported	TPARITY per byte (optional)
Power management	Limited	Enhanced via TWAKEUP
Data integrity	CRC/checksum in TUSER	Built-in parity option

4.6.2 Parity Check Location

Key difference from AXIS5 Master: - **Slave:** Parity checked on **input** (`s_axis_tdata` vs `s_axis_tparity`) - **Master:** Parity checked on **output** (`m_axis_tdata` vs `m_axis_tparity`)

Checking at the input means transmission errors get caught at the receiving side, as early as possible.

4.6.3 Skid Buffer Sizing

- **Typical depth:** 4-8 entries
- Deeper buffers:
  - Absorb longer backend stalls
  - Increase area and latency
- Shallower buffers:
  - Lower latency
  - May cause upstream backpressure

Recommendation: depth 4 covers most applications; go deeper if your backend stalls frequently.

4.6.4 Parity Implementation

When ENABLE_PARITY=1: - **Overhead:** 1 bit per data byte, so 12.5% extra wires regardless of bus width (a 512-bit bus adds 64 parity bits) - **Detection:** Odd numbers of bit errors within a byte (even parity); two flipped bits in the same byte are undetected - **Correction:** None - error flag only - **Polarity:** Even parity only; there is no parameter to select odd parity - **Use case:** Low-cost error detection in reliable environments

For stronger protection, use: - CRC in TUSER field - ECC (external module) - End-to-end checksums at packet level

4.6.5 Error Handling Strategy

The parity_error flag is **sticky** (latches high until reset): - Simplifies error detection (level-sensitive interrupt) - Software must clear by resetting module - For edge-sensitive interrupts, external logic must detect rising edge

Alternative: modify the module to add a parity_error_clear input if your system needs one.

4.7 Related Modules

- **AXIS5 Master** - AXIS5 master interface (transmit side)
- **AXIS5 Slave CG** - Clock-gated variant with power management
- **AXIS5 Master CG** - Clock-gated master variant
- **AXIS4 Slave** - AXIS4 version for comparison
- **AMBA5 Overview** - AMBA5 specifications and extensions

4.8 Testing

val/amba/test_axis5_slave.py exercises this module. It collects 3 parameter cases at the default REG_LEVEL.

```
source env_python
pytest val/amba/test_axis5_slave.py -v
```

4.9 Navigation

- **← Back to AXIS5 Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**

5 AXIS5 Slave with Clock Gating

Module: `axis5_slave_cg.sv` **Location:** `rtl/amba/axis5/` **Status:** Production Ready

5.1 Overview

The AXIS5 Slave CG (Clock Gated) wraps the standard AXIS5 slave with the AMBA clock gate controller, giving you an AXI5-Stream slave interface that gates its own clock during idle periods. Full AXIS5 functionality survives the wrapping — wake-up signaling, optional parity, the skid buffer, all of it.

5.1.1 Key Features

- Same AXI5-Stream data path as `axis5_slave` (see its signal coverage note; TKEEP and TPOISON are not implemented)
- Automatic clock gating during idle periods for power savings
- TWAKEUP: Wake-up signaling integration with clock gate control
- TPARITY: Optional parity protection, 1 bit per byte, with error detection (proprietary extension)
- Configurable idle count threshold for gating activation
- Internal skid buffer for backpressure handling
- Parity error detection on input side
- Clock gating status output

5.1.2 Power Management Features

Feature	Implementation	Benefit
Clock gating	Automatic via idle detection	Reduces dynamic power
Wake-up integration	TWAKEUP prevents gating	Preserves responsiveness
Configurable threshold	Programmable idle count	Tune power vs. latency
Activity detection	Multi-source OR logic	Comprehensive coverage

5.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH	int	4	Internal skid buffer depth
AXIS_DATA_WIDTH	int	32	AXIS data bus width (must be multiple of 8)
AXIS_ID_WIDTH	int	8	AXIS ID signal width (0 to disable)
AXIS_DEST_WIDTH	int	4	AXIS TDEST signal width (0 to disable)
AXIS_USER_WIDTH	int	1	AXIS TUSER signal width (0 to disable)
ENABLE_WAKEUP	bit	1	Enable TWAKEUP signal (1=enabled)
ENABLE_PARITY	bit	0	Enable TPARITY signal (1=enabled)
CG_IDLE_COUNT_WIDTH	int	4	Clock gate idle counter width

Note on `ENABLE_WAKEUP`: the default is 1, so TWAKEUP ports exist and are carried through the buffer unless you explicitly set `ENABLE_WAKEUP=0`. Set it to 0 for an AXI4-Stream-compatible port list with no wake-up sideband and slightly less area. `ENABLE_PARITY` defaults to 0 because TPARITY is a proprietary extension.

5.2.1 Derived Values (do not override)

These are declared in the parameter list so they can be used in port widths, but they are derived from the parameters above. Overriding them directly produces an inconsistent module.

Name	Derivation	Meaning
DW	AXIS_DATA_WIDTH	Data width short name
IW	AXIS_ID_WIDTH	ID width short name
DESTW	AXIS_DEST_WIDTH	DEST width short name
UW	AXIS_USER_WIDTH	USER width short name
SW	DW/8	Strobe width in bytes
PW	SW	Parity width - 1 bit per byte

Name	Derivation	Meaning
ICW	CG_IDLE_COUNT_WIDTH	Idle count width short name
IW_WIDTH / DESTW_WIDTH / UW_WIDTH	max(width, 1)	Zero-width avoidance for disabled sidebands
PW_WIDTH	ENABLE_PARITY ? PW : 1	TPARITY port width

5.3 Ports

5.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXIS ungated clock (always running)
aresetn	1	Input	AXIS active-low asynchronous reset

5.3.2 Clock Gating Configuration

Port	Width	Direction	Description
i_cg_enable	1	Input	Clock gating enable (1=allow gating, 0=force always on)
i_cg_idle_count	ICW	Input	Idle clock cycles before gating activates

5.3.3 Slave AXIS5 Interface (Input Side)

Port	Width	Direction	Description
s_axis_tdata	DW	Input	Transfer data from upstream
s_axis_tstrb	SW	Input	Transfer byte strobes
s_axis_tlast	1	Input	Last transfer in packet

Port	Width	Direction	Description
s_axis_tid	IW_WIDTH	Input	Transfer ID (optional)
s_axis_tdest	DESTW_WIDTH	Input	Transfer destination (optional)
s_axis_tuser	UW_WIDTH	Input	Transfer user-defined signals (optional)
s_axis_tvalid	1	Input	Transfer valid from upstream
s_axis_tready	1	Output	Transfer ready (skid buffer not full)
s_axis_twakeup	1	Input	Wake-up signal (prevents clock gating)
s_axis_tparity	PW_WIDTH	Input	Data parity per byte (AXIS5 extension)

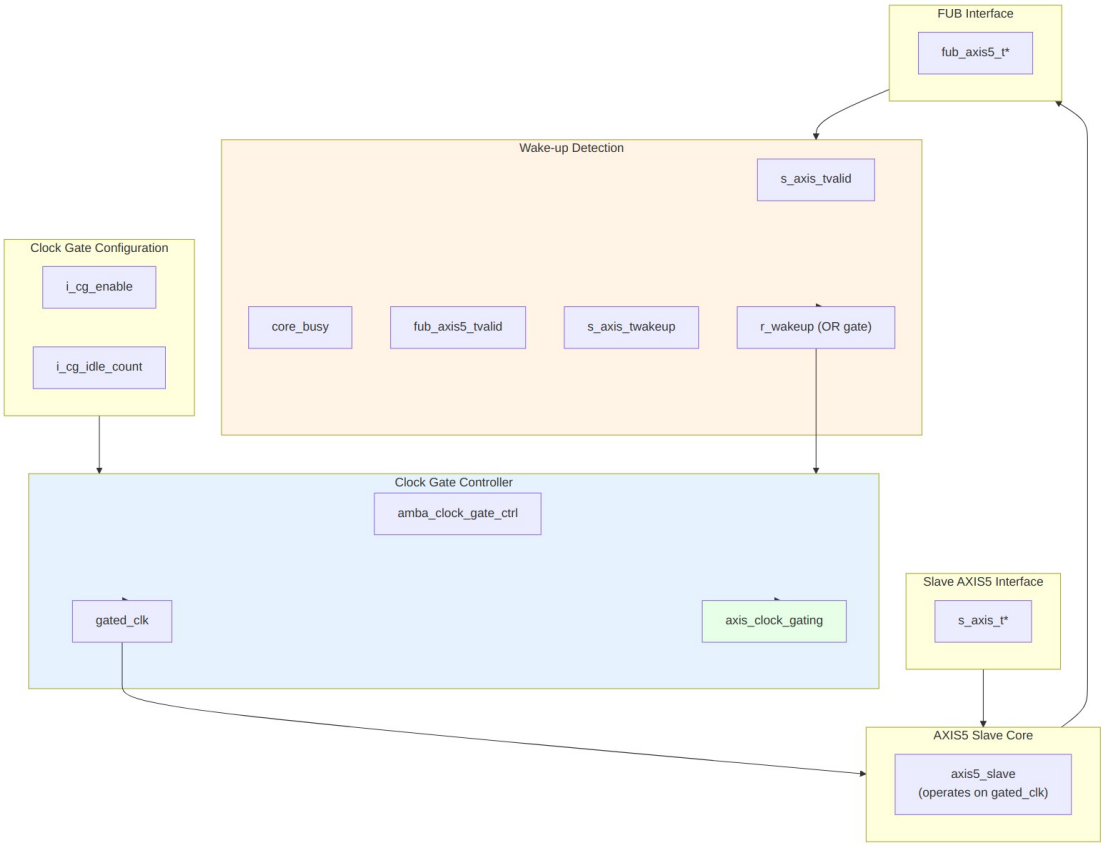
5.3.4 FUB AXIS5 Interface (Output Side to Backend)

Port	Width	Direction	Description
fub_axis5_tdata	DW	Output	Transfer data to backend
fub_axis5_tstrb	SW	Output	Transfer byte strobes
fub_axis5_tlast	1	Output	Last transfer in packet
fub_axis5_tid	IW_WIDTH	Output	Transfer ID (optional)
fub_axis5_tdest	DESTW_WIDTH	Output	Transfer destination (optional)
fub_axis5_tuser	UW_WIDTH	Output	Transfer user-defined signals (optional)
fub_axis5_tvalid	1	Output	Transfer valid to backend
fub_axis5_tready	1	Input	Transfer ready from backend
fub_axis5_t	1	Output	Wake-up signal

Port	Width	Direction	Description
wakeup			
fub_axis5_tparity	PW_WIDTH	Output	Data parity per byte (AXIS5 extension)

5.3.5 Status Outputs

Port	Width	Direction	Description
busy	1	Output	Module busy (data in buffer or input valid)
parity_error	1	Output	Parity error detected on input (sticky flag)
axis_clock_gating	1	Output	Clock gating active status (1=gated, 0=running)



Functional Description

5.3.6 Clock Gating Control

Wake-up detection logic:

```
r_wakeup <= s_axis_tvalid ||    // Input has data
            core_busy ||         // Core processing
            fub_axis5_tvalid ||  // Backend has data
            s_axis_twakeup;      // Explicit wake-up
```

The clock gate controller: 1. Monitors `r_wakeup` for activity 2. If idle (r_wakeup=0) for `i_cg_idle_count` cycles, gates the clock 3. On activity (r_wakeup=1), ungates the clock 4. Respects `i_cg_enable` (clock forced on when 0)

5.3.7 Wake-up and Gating Latency

Activity is registered once (AXI4, AXI5, AXI4-Lite, AXI4-Stream) or twice (APB, APB5, AXI5-Stream – with one exception: `apb4_slave_cdc_cg` drives `amba_clock_gate_ctrl` combinationally and so registers once, not twice) before reaching the ICG enable, which is combinational. AXI5-Stream is a **two-stage** family: `r_wakeup` in this wrapper, then a second `r_wakeup` inside `amba_clock_gate_ctrl`. The first gated-clock rising edge available to the core therefore arrives 3 `aclk` cycles after activity asserts. Budget accordingly:

Event	Latency	Path
Activity to first usable gated-clock edge	3 `aclk` cycles (2 register stages)	`s_axis_tvalid` to `r_wakeup` to controller `r_wakeup` to combinational ICG enable to next edge
Last activity to clock gated	`i_cg_idle_count` + 3 `aclk` cycles	`i_cg_idle_count` + 1 after the internal wakeup deasserts, plus two wake-up register stages

The wake-up path is **not** zero-latency. Nothing is lost during those cycles: `s_axis_tready` is driven by the skid buffer on `gated_clk`, so it stays low while the clock is stopped and the upstream master simply holds TVALID until the clock resumes. Both registers and the idle counter run on the ungated `aclk`, so the wake-up path stays live even while the core clock is stopped.

5.3.8 Reset Behavior

Reset is safe with respect to gating; the clock is guaranteed to be running whenever reset is asserted:

- `aresetn` is asynchronous and is passed **ungated** to both the clock gate controller and the core, so its assertion edge does not depend on the gated clock.
- `r_wakeup` in this wrapper and `r_wakeup` inside `amba_clock_gate_ctrl` both reset to 1, which forces the ICG enable active. The clock therefore runs during reset and for at least the idle countdown after reset deassertion.
- `clock_gate_ctrl` loads its idle counter with `i_cg_idle_count` on reset, so gating cannot re-engage until a full idle period has elapsed after release.

Because reset removal is asynchronous, apply the usual practice of releasing `aresetn` synchronously to `aclk` at the system level.

5.3.9 Power Saving Mechanism

Clock gating states:

Condition	r_wakeup	Clock State	Power State
Receiving data	1	Running	Full power
Buffer has data	1	Running	Full power
Backend processing	1	Running	Full power
Wake-up asserted	1	Running	Full power
Idle < threshold	0	Running	Full power
Idle ≥ threshold	0	Gated	Low power

Benefits: - Reduces dynamic power during idle periods - Wake-up on new transfers costs 2 register stages; the first usable gated-clock edge arrives 3 `aclk` cycles after activity (see Wake-up and Gating Latency) - No protocol impact (transparent to upstream/backend)

5.3.10 Parity Error Detection

When `ENABLE_PARITY=1`: - Parity checked on **input side** (`s_axis_tdata` vs `s_axis_tparity`) - Early error detection at receive interface - `parity_error` flag is sticky (latches until reset) - No transfer can slip past the check while gated: `s_axis_tready` is driven on `gated_clk`, so no transfer is accepted until the clock has resumed

5.3.11 Idle Count Configuration

Typical values for `i_cg_idle_count`: - **Aggressive power saving:** 2-4 cycles - Quick gating, may have frequent gate/ungate transitions - Best for bursty traffic with long idle periods - **Balanced:** 8-16 cycles - Reduces gate/ungate overhead - Good for moderate traffic patterns - **Conservative:** 32-64 cycles - Only gates during extended idle - Minimal gate/ungate overhead

Trade-off: lower count means more power savings but more gate transitions, and each transition has its own dynamic overhead.

5.4 Timing Characteristics

5.4.1 Clock Gating Activation on Slave

Timing diagram pending. The signals and sequence this scenario exercises:

- aclk (always running)
- s_axis_tvalid (input activity)
- r_wakeup (activity detection)
- i_cg_idle_count (threshold)
- gated_clk (starts gating after threshold)
- axis_clock_gating (status)
- New s_axis_tvalid arrival ungates clock

5.4.2 Wake-up Preventing Gating During Receive

Timing diagram pending. The signals and sequence this scenario exercises:

- aclk
- s_axis_twakeup (asserted during transfer)
- s_axis_tvalid
- r_wakeup (stays high)
- gated_clk (remains running)
- axis_clock_gating (stays 0)
- fub_axis5_tvalid (output forwarded)

5.4.3 Backend Backpressure with Clock Gating

Timing diagram pending. The signals and sequence this scenario exercises:

- aclk
- s_axis_tvalid (upstream sending)
- fub_axis5_tready (backend blocked)
- core_busy (stays high)
- r_wakeup (stays high)
- gated_clk (remains running due to busy)
- axis_clock_gating (stays 0 during backpressure)

5.5 Usage Examples

5.5.1 Basic Clock-Gated Slave

```
axis5_slave_cg #(
    .SKID_DEPTH             (4),
    .AXIS_DATA_WIDTH        (64),
    .AXIS_ID_WIDTH          (8),
    .AXIS_DEST_WIDTH        (4),
    .AXIS_USER_WIDTH        (1),
    .ENABLE_WAKEUP          (1),
    .ENABLE_PARITY          (0),
    .CG_IDLE_COUNT_WIDTH    (4)
) u_axis5_slave_cg (
    .aclk                   (axis_clk),
    .aresetn                (axis_rst_n),

    // Clock gating configuration
    .i_cg_enable            (1'b1),        // Enable clock gating
    .i_cg_idle_count        (4'd8),        // Gate after 8 idle cycles

    // Slave interface (from upstream)
    .s_axis_tdata           (s_axis_tdata),
    .s_axis_tstrb           (s_axis_tstrb),
    .s_axis_tlast           (s_axis_tlast),
    .s_axis_tid             (s_axis_tid),
    .s_axis_tdest           (s_axis_tdest),
    .s_axis_tuser           (s_axis_tuser),
    .s_axis_tvalid          (s_axis_tvalid),
    .s_axis_tready          (s_axis_tready),
    .s_axis_twakeup         (s_axis_twakeup),
```

```
    .s_axis_tparity         (8'h00),

    // FUB interface (to backend)
    .fub_axis5_tdata        (fub_tdata),
    .fub_axis5_tstrb        (fub_tstrb),
    .fub_axis5_tlast        (fub_tlast),
    .fub_axis5_tid          (fub_tid),
    .fub_axis5_tdest        (fub_tdest),
    .fub_axis5_tuser        (fub_tuser),
    .fub_axis5_tvalid       (fub_tvalid),
    .fub_axis5_tready       (fub_tready),
    .fub_axis5_twakeup      (fub_twakeup),
    .fub_axis5_tparity      (),

    // Status
    .busy                   (axis_busy),
    .parity_error           (),
    .axis_clock_gating      (axis_clk_gated)  // Monitor gating status
);
```

5.5.2 With Parity Protection and Power Monitoring

```
// Clock-gated slave with parity and comprehensive monitoring
axis5_slave_cg #(
    .AXIS_DATA_WIDTH        (32),
    .ENABLE_WAKEUP          (1),
    .ENABLE_PARITY          (1),  // Enable parity checking
    .CG_IDLE_COUNT_WIDTH    (4)
) u_rx_slave_cg (
    .aclk                   (sys_clk),
    .aresetn                (sys_rst_n),

    // Dynamic clock gating control
    .i_cg_enable            (cfg_cg_enable),
    .i_cg_idle_count        (cfg_cg_threshold),

    // Slave interface with parity
    .s_axis_tdata           (rx_tdata),
    .s_axis_tstrb           (rx_tstrb),
    .s_axis_tlast           (rx_tlast),
    .s_axis_tid             (rx_tid),
    .s_axis_tdest           (rx_tdest),
    .s_axis_tuser           (rx_tuser),
    .s_axis_tvalid          (rx_tvalid),
    .s_axis_tready          (rx_tready),
    .s_axis_twakeup         (rx_twakeup),
    .s_axis_tparity         (rx_tparity),  // 4 bits for 32-bit data
```

```
    // FUB interface
    .fub_axis5_tdata        (proc_tdata),
    .fub_axis5_tstrb        (proc_tstrb),
    .fub_axis5_tlast        (proc_tlast),
    .fub_axis5_tid          (proc_tid),
    .fub_axis5_tdest        (proc_tdest),
    .fub_axis5_tuser        (proc_tuser),
    .fub_axis5_tvalid       (proc_tvalid),
    .fub_axis5_tready       (proc_tready),
    .fub_axis5_twakeup      (proc_twakeup),
    .fub_axis5_tparity      (proc_tparity),

    // Status monitoring
    .busy                   (rx_busy),
    .parity_error           (rx_parity_err),
    .axis_clock_gating      (rx_clk_gated)
);

// Error and power monitoring
always_ff @(posedge sys_clk or negedge sys_rst_n) begin
    if (!sys_rst_n) begin
        parity_error_count <= '0;
        total_cycles <= '0;
        gated_cycles <= '0;
    end else begin
        // Count cycles
        total_cycles <= total_cycles + 1;
        if (rx_clk_gated)
            gated_cycles <= gated_cycles + 1;

        // Count parity errors
        if (rx_parity_err && !prev_parity_err)
            parity_error_count <= parity_error_count + 1;
        prev_parity_err <= rx_parity_err;
    end
end

// Calculate power savings
assign gated_duty_pct = (gated_cycles * 100) / total_cycles;
```

Read this metric carefully. `gated_duty_pct` is the fraction of time the clock was gated, not a power saving. Actual dynamic power saved depends on the switching activity that would have occurred during those cycles; leakage is unaffected by gating; and the gating logic and its clock-tree branch consume power themselves. Use the number as a gating-effectiveness indicator, and get real figures from a power analysis tool fed a VCD or SAIF from a representative workload.

5.5.3 Receive Pipeline with Power Management

```
// Network receive path with clock gating
axis5_slave_cg #(
    .AXIS_DATA_WIDTH        (512),  // Wide data path
    .AXIS_ID_WIDTH          (4),
    .ENABLE_WAKEUP          (1),
    .ENABLE_PARITY          (1),
    .CG_IDLE_COUNT_WIDTH    (6)     // Wider counter for longer idle
) u_rx_interface (
    .aclk                   (net_clk),
    .aresetn                (net_rst_n),

    // Aggressive clock gating for power
    .i_cg_enable            (1'b1),
    .i_cg_idle_count        (6'd4),  // Gate quickly after 4 idle
cycles

    // From network PHY
    .s_axis_tdata           (phy_rx_tdata),
    .s_axis_tstrb           (phy_rx_tstrb),
    .s_axis_tlast           (phy_rx_tlast),
    .s_axis_tid             (phy_rx_tid),
    .s_axis_tdest           (phy_rx_tdest),
    .s_axis_tuser           (phy_rx_tuser),
    .s_axis_tvalid          (phy_rx_tvalid),
    .s_axis_tready          (phy_rx_tready),
    .s_axis_twakeup         (phy_rx_twakeup),
    .s_axis_tparity         (phy_rx_tparity),

    // To packet processor
    .fub_axis5_tdata        (pkt_proc_tdata),
    .fub_axis5_tstrb        (pkt_proc_tstrb),
    .fub_axis5_tlast        (pkt_proc_tlast),
    .fub_axis5_tid          (pkt_proc_tid),
    .fub_axis5_tdest        (pkt_proc_tdest),
    .fub_axis5_tuser        (pkt_proc_tuser),
    .fub_axis5_tvalid       (pkt_proc_tvalid),
    .fub_axis5_tready       (pkt_proc_tready),
    .fub_axis5_twakeup      (pkt_proc_twakeup),
    .fub_axis5_tparity      (pkt_proc_tparity),

    // Status for system monitoring
    .busy                   (rx_busy),
    .parity_error           (rx_parity_err),
    .axis_clock_gating      (rx_clk_gated)
);
```

```
// Interrupt on parity error
assign parity_err_irq = rx_parity_err && !prev_rx_parity_err;
```

5.6 Design Notes

5.6.1 Clock Gating vs. Non-Gated Variant

Aspect	axis5_slave	axis5_slave_cg
Area	Smaller	+5-10% (clock gate logic)
Dynamic power	Higher	Lower (gating reduces)
Static power	Same	Same
Latency	Lower	Same in steady state; +3 `aclk` cycles when waking from a gated state (two register stages)
Complexity	Lower	Higher (gating control)
Use case	Always-on systems	Power-sensitive designs

When to use clock-gated variant: - Battery-powered devices - Receive paths with bursty traffic - Power budget constraints - SoC integration with power domains

5.6.2 Slave-Specific Gating Considerations

Differences from master variant: 1. **Wake-up source:** Upstream signal (`s_axis_twakeup`) 2. **Activity detection:** Input valid (`s_axis_tvalid`) 3. **Backpressure:** Backend ready affects gating

Receive path characteristics: - Often idle waiting for upstream data - Good candidate for aggressive clock gating - Wake-up from upstream provides early notification

5.6.3 Gate Transition Overhead

Each clock gate transition has overhead: - **Gate activation:** `i_cg_idle_count` + 1 cycles after the internal wakeup deasserts, which is `i_cg_idle_count` + 3 cycles after the last bus activity - **Ungate activation:** two wake-up register stages; first usable gated-clock edge 3 cycles after activity asserts - **Dynamic power:** Gate/ungate switching consumes power

Recommendation: set `i_cg_idle_count` high enough to amortize the transition overhead, especially in scenarios where gating engages frequently.

5.6.4 Parity Error Handling with Clock Gating

Parity errors detected on input side: - The comparison is combinational on `s_axis_tdata` and `s_axis_tparity`, so it is evaluated before data enters the skid buffer - The `parity_error` register is clocked by `gated_clk` and captures on an accepted transfer (`s_axis_tvalid && s_axis_tready`). No error can be missed while gated, because `s_axis_tready` is itself driven on `gated_clk` and stays low until the clock resumes, and `s_axis_tvalid` wakes the clock first - Once set, the flag holds through subsequent gating; a stopped clock cannot clear a register

Important: the `parity_error` flag requires reset to clear — plan for error recovery.

5.6.5 Clock Domain Considerations

- **Input signals:** Sampled by the wake-up logic on `aclk` (always running) and by the core on `gated_clk`
- **Core logic:** Operates on `gated_clk` domain
- **Output signals:** Driven by `gated_clk` domain
- **Status outputs:** `axis_clock_gating` on `aclk` domain

No CDC synchronizers are needed, because `gated_clk` is `aclk` passed through an ICG cell: same frequency, same phase, no independent clock source. There is still physical work to do:

- **Clock tree synthesis:** `gated_clk` must be balanced against `aclk`. The wake-up registers and `axis_clock_gating` sit on `aclk` while the core sits on `gated_clk`, so uncontrolled skew between the two branches eats directly into the timing budget on every path that crosses between them.
- **ICG placement:** place the ICG cell close to the root of the gated branch so the insertion delay it adds is common to the whole core.
- **FPGA targets:** `clock_gate_ctrl` instantiates an ICG cell, which is an ASIC library primitive. On FPGAs use a clock-enable style implementation or a device buffer with an enable (for example `BUFGCE`) instead; a bare ICG will not map cleanly.

5.7 Related Modules

- **AXIS5 Slave** - Base AXIS5 slave without clock gating
- **AXIS5 Master CG** - Clock-gated master variant
- **AXIS5 Master** - Base AXIS5 master
- **AMBA Clock Gate Controller** - Clock gating controller
- **AMBA5 Power Management** - AMBA5 power features

5.8 Testing

`val/amba/test_axis5_slave_cg.py` exercises this module. It collects 3 parameter cases at the default `REG_LEVEL`.

```
source env_python
pytest val/amba/test_axis5_slave_cg.py -v
```

5.9 Navigation

- **← Back to AXIS5 Index**
- **← Back to rtl-amba Index**
- **← Back to Main Documentation Index**